

MSBD5014A Project Report

Personalized Content Hub with Agentic RAG and Dynamic Data Integration
(Subtopic: News Recommendation Agent)

CHENG Tsz Ying (21126455)

Abstract

This project develops a personalized news recommendation system leveraging a large language model (LLM) and Retrieval-Augmented Generation (RAG) to deliver tailored news articles based on user preferences. Implemented as a single-user web application, the system supports natural language queries, enabling intuitive interaction through a chat interface. Key features include a workspace system for managing multiple interest profiles, bookmarking and quoting for enhanced user engagement, and a web search for accessing news older than one week. The system fetches technology news daily, stores summaries in an RAG database, and updates user interest profiles based on interactions to improve recommendation accuracy. The responsive interface ensures seamless use across devices. This implementation demonstrates an effective approach to personalized content delivery, with the potential for future enhancements in scalability and profile accuracy. The project code is provided on <https://github.com/christycty/rag-news-agent>.

1 Introduction

The rapid growth of online news creates an overwhelming volume of information, making it challenging for users to find relevant and personalized content. This project addresses this issue by developing a news recommendation system that delivers tailored news articles based on individual user preferences. Implemented as a single-user web application, the system utilizes a large language model (LLM) and Retrieval-Augmented Generation (RAG) to process natural language queries and retrieve pertinent news from a dynamic database. Users interact through an intuitive chat interface, receiving personalized recommendations that adapt to their interests over time. Key features include workspace profiles for topic-specific browsing, bookmarking for saving articles, and web searches for older content.

This report is organized by feature to highlight the system's functionality and implementation. Following a review of related work, the design, scope, and technical implementation of each feature are discussed.

2 Related Works

Personalized news recommendation has progressed from traditional feature-based methods to sophisticated deep learning approaches [1]. Early systems employed information filtering, matching articles to user preferences using textual or semantic features [2, 3], or collaborative filtering, identifying similar user preferences through user-item interactions [4, 5, 6]. These methods, though effective, depended on rigid inputs like explicit ratings or predefined topics, constraining flexibility [1]. For example, contextual bandits [7] and semantic analysis [8] enhanced content-based recommendations, while matrix factorization [9] and large-scale clustering [4] improved collaborative filtering.

Later models adopted complex architectures to capture user-news interactions. Embedding-based approaches [10], knowledge graphs [11, 12], and attention mechanisms [13, 14] model dynamic user interests, while reinforcement learning [15] adapts to evolving behaviors, offering more nuanced recommendations.

Recent studies leverage large language models (LLMs) for their superior semantic understanding and knowledge integration [16]. LLMs enhance personalization by capturing news semantics [17, 18, 19], generating category descriptions [20], interpreting natural language queries [21], and producing coherent narratives from recommended articles [22]. Retrieval-Augmented Generation (RAG) further improves LLMs by dynamically retrieving external knowledge for recommendations [23, 24]. Agentic RAG optimizes this process by autonomously determining retrieval needs, enhancing query responsiveness [25].

3 Overview

This section describes the structure and functionality of a news recommendation system implemented as a single-user web application. The system uses a large language model (LLM) and Retrieval-Augmented Generation (RAG) to process user queries and recommend technology news articles. It outlines the system’s objectives, features, and design, providing a foundation for detailed discussions in later sections.

3.1 System Goals and Scope

The system is designed to recommend news articles based on user preferences, using RAG for article retrieval and an LLM for query processing and ranking. Its objectives are:

- Implement personalized recommendations using RAG and dynamic interest profiles.
- Process natural language queries with an LLM-based agent.
- Support retrieval of recent and older news via RAG and web searches.

The system is a single-user, locally hosted application that focuses on recommendation algorithms. It stores news for one week to prioritize recency, with web searches for older content. The scope excludes multi-user support or cloud deployment to simplify development.

3.2 Features

The system’s features, shown in Figure 1, enable news recommendation and user interaction. The figure illustrates interactions between the frontend query module, RAG database, and LLM agent, with core features in black and enhanced features in blue. Core features form the recommendation pipeline, while enhanced features extend functionality.

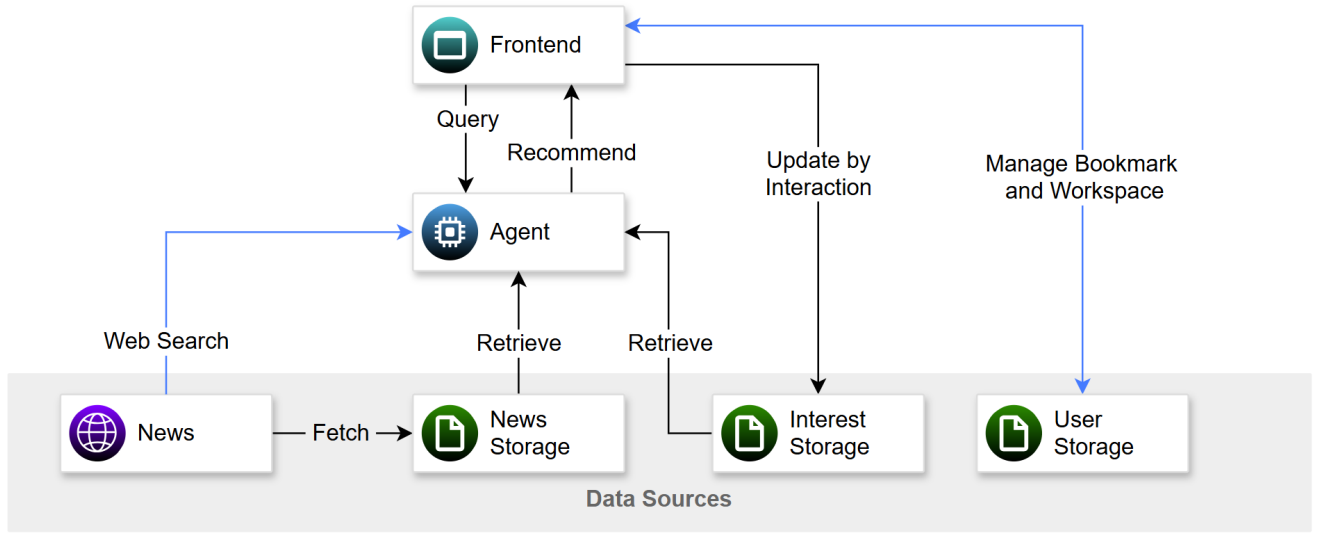


Figure 1: Feature Overview.

3.2.1 Core Features

- **Personalized News Recommendation Based on RAG and LLM:** The system fetches technology news daily and stores summaries in a RAG database. The LLM retrieves articles using similarity search and ranks them based on a user’s interest profile, updated via interactions (e.g., clicks, bookmarks).
- **Natural Language Queries:** Users submit queries in natural language, processed by the LLM to retrieve relevant articles. This supports follow-up questions, unlike traditional systems requiring predefined topics or search strings.

3.2.2 Extra Features

- **Web Search:** For queries about news older than one week, the LLM triggers a web search. Results are combined with RAG-retrieved articles, with duplicates filtered by URL.
- **Workspace System:** Users create workspaces with independent interest profiles and bookmarks, enabling topic-specific recommendations (e.g., machine learning, healthcare).

- **User News Management:** Users bookmark articles for later access or quote them in queries for related recommendations. Bookmarks update the interest profile, and quoting provides query context.

3.3 System Design

The system is a web application with a frontend for user interaction and a backend for processing. The frontend includes three pages: chat for queries, profile for workspace management, and bookmark for saved articles, ensuring modular functionality.

3.3.1 System Architecture

The backend, shown in Figure 2, includes modules for news fetching, recommendation, and storage. The RAG database stores article embeddings for retrieval, while interest profiles and bookmarks use separate databases. The tech stack, depicted in blue in Figure 2, consists of React and Material Tailwind for the multi-page frontend, Python for backend processing, FastAPI for API endpoints, ChromaDB for the RAG database, SQLite for bookmarks and workspaces, and HuggingFace models for generating article embeddings. These technologies enable efficient query processing, vector-based retrieval, and data management.

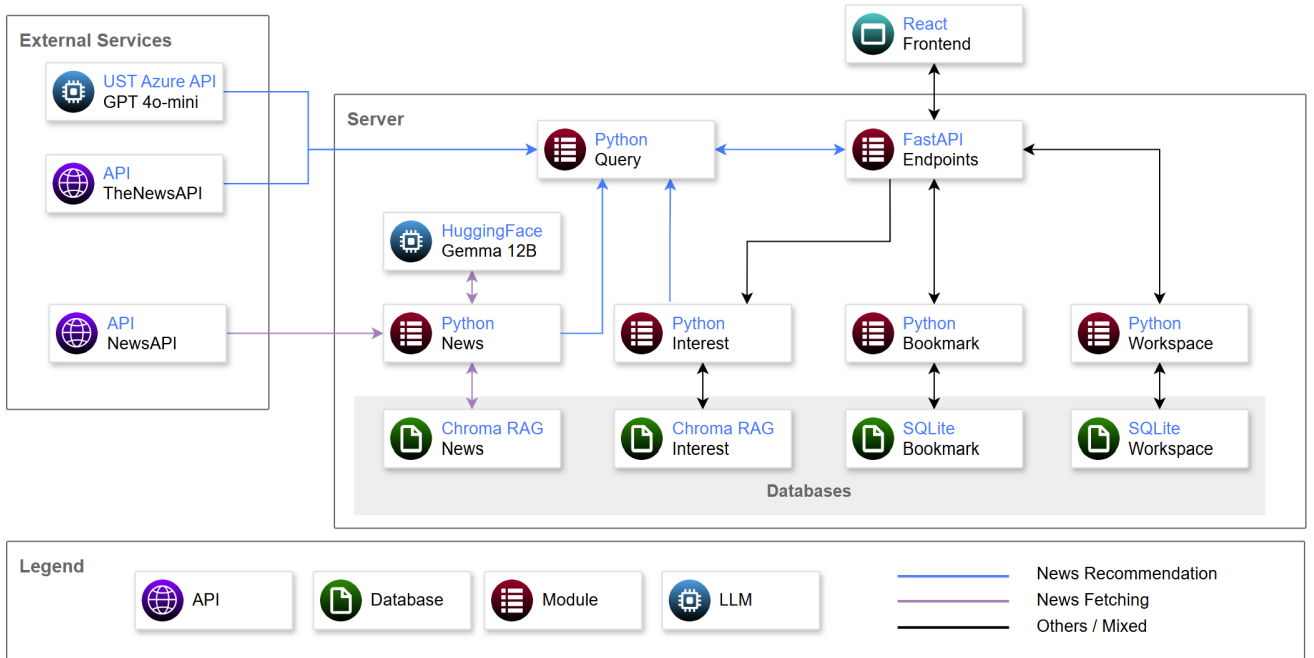


Figure 2: System architecture.

3.3.2 Communication

API endpoints facilitate data exchange between frontend and backend, as illustrated in Figure 3. Most calls correspond to specific modules: the chat page sends queries to the query module, the

profile page manages workspaces via the workspace module, and the bookmark page handles saved articles through the bookmark module. On app load, the system fetches workspace and user data, independent of specific pages, to initialize the interface.

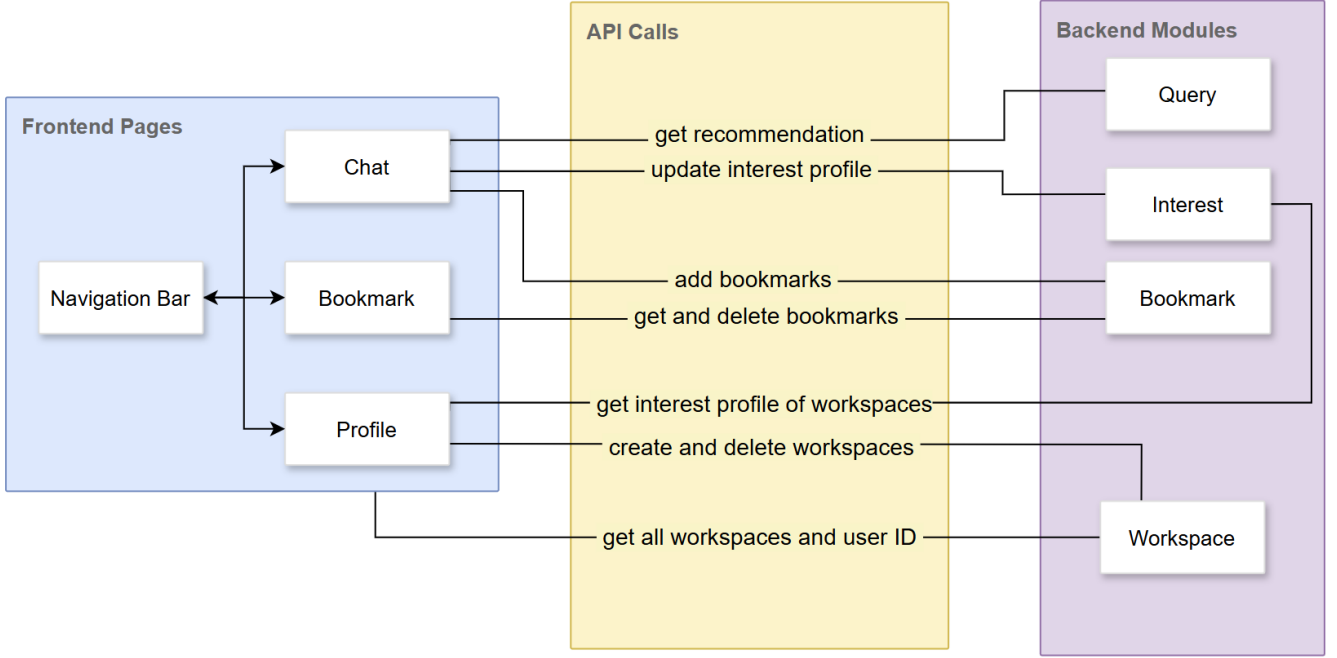


Figure 3: API call between frontend and server.

4 News Management

This section outlines the pipeline for acquiring, processing, and storing technology news articles in a database, managed by the news module in Figure 4. Executed daily using a cronjob, the process retrieves fresh articles, processes them for recommendation, and removes entries older than one week to ensure timeliness. The pipeline includes fetching full articles, generating summaries and tags, and storing data in a vector database. Additionally, a web search mechanism serves as a supplementary data source for older news, activated during recommendation (Section 5).

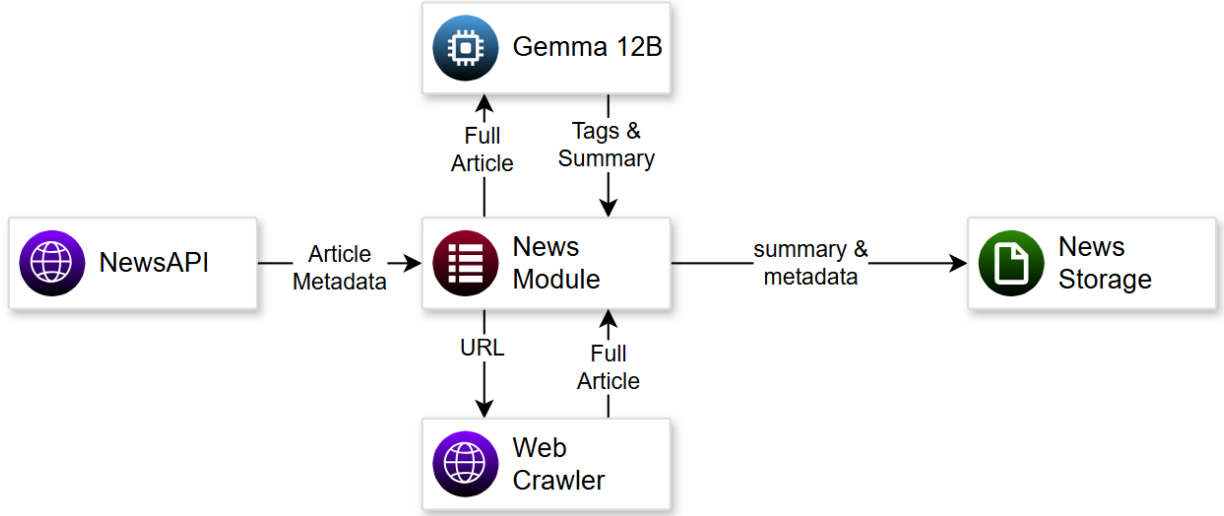


Figure 4: News management pipeline.

4.1 Fetch Full Article

The news module retrieves metadata for English technology articles published in the past 24 hours using NewsAPI [26]. Metadata includes titles, descriptions, URLs, publication dates, sources, and authors, with each API call yielding up to 20 articles. Fetching continues until no new articles are available or the API rate limit is reached.

Articles are processed and stored sequentially to ensure better error handling, so early program termination due to failures does not affect prior entries. Duplicates are skipped by matching URLs against the database. A web crawler extracts full article text from each URL, discarding inaccessible paywalled articles.

4.2 Generate Summary and Tags

Full articles are processed by a large language model (LLM) to generate 150-word summaries and tags using the prompt in Appendix 9.1.1. Storing summaries instead of full texts offers users a concise news overview in the interface and minimizes storage requirements while providing sufficient content for Retrieval-Augmented Generation (RAG).

Tags are automatically generated to represent key topics of the articles, supporting user interest profiles and enhancing recommendation precision. The use of LLMs for tag generation minimizes manual effort and domain-specific knowledge to incorporate tags into recommendation models [20]. The LLM’s response is parsed using regex to extract summaries and tags, with tags trimmed, lowercased, and lemmatized using NLTK’s WordNet lemmatizer [27] to reduce redundancy.

4.3 Storage

Processed articles are stored in ChromaDB, a vector database optimized for text similarity retrieval [28]. Each entry comprises a document (summary), metadata (title, description, URL, fetch time, publication date, source, tags), and a UUID4 identifier. Summaries are converted into embeddings using HuggingFace language models [29], fine-tuned for text-to-embedding tasks, enabling efficient RAG-based retrieval during recommendation.

4.3.1 Web Search

For recommendation queries requiring news older than one week, a web search is performed using TheNewsAPI [30]. Unlike NewsAPI, which provides metadata for recent articles, TheNewsAPI retrieves full-text articles up to one year old, bypassing scraping (Table 1). Executed during recommendation, the LLM generates search queries based on user input and interest profiles, with results filtered for relevance. This supplementary data source extends the system’s coverage, supporting historical queries and enhancing recommendation accuracy.

Feature	NewsAPI	TheNewsAPI
Search Period	1 month	1 year
Articles per Request	20	3
Daily Request Count	100	100
Return Content	Metadata only	Metadata and Full Text
Appropriate Task	News Fetch	Web Search

Table 1: Comparison of the news API services.

5 News Recommendation

This section presents the personalized news recommendation system, leveraging a large language model (LLM) and Retrieval-Augmented Generation (RAG) to deliver relevant articles based on natural language queries. Unlike traditional systems reliant on predefined topics [1], it supports dynamic user interest profiles and a chat interface, with web searches for older news. The recommendation pipeline, illustrated in Figure 7, comprises user interest profiles, chat interface, query processing, and ranking with response generation.

5.1 Interest Profile

Each workspace maintains an independent interest profile, comprising tags with scores reflecting user preferences, stored in ChromaDB [28]. Scores are updated via FastAPI frontend requests triggered by user interactions. Clicking an article’s link, a widely adopted feature for user modeling [31, 32, 33], increments tag scores to capture engagement. Consequently, bookmarking an article triggers a higher score increment, reflecting stronger user interest. As a single-user system, the

recommendation process relies exclusively on content-based features derived from tags, making methods such as collaborative filtering [4] or neural network-based user modeling infeasible.

To simplify implementation, stay-time tracking was omitted, and tag updates were restricted to exact matches, avoiding slow similarity searches that increased response times to 10 seconds. This design ensures near-instant updates, supporting real-time personalization in query processing and ranking. By prioritizing efficiency, the system maintains accurate profiles without complex threading, enhancing recommendation relevance based on user engagement.

5.2 User Interface

The chat interface, shown in Figure 5, adopts a familiar design inspired by messaging applications. Users and the LLM-based agent exchange messages displayed as dialogue bubbles, with a scrollable history for reviewing past interactions. Users enter queries via a text box at the bottom, and each agent response includes three recommended articles with a concise summary addressing the query. This conversational approach supports question-answering, distinguishing the system from traditional news recommenders that focus solely on article retrieval.

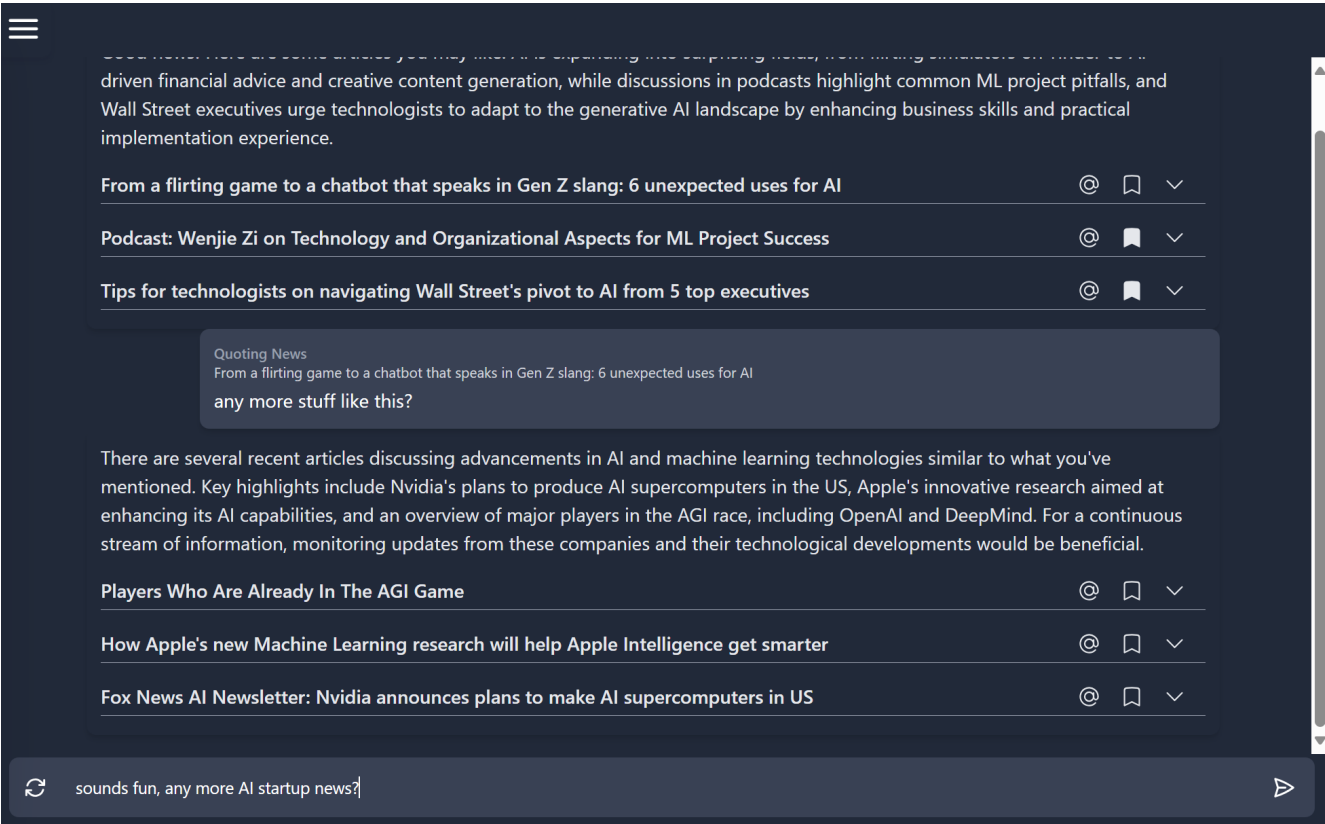


Figure 5: Chat user interface.

5.2.1 Dialogue Flow

The session-based dialogue flow, illustrated in Figure 6, begins with a daily recommendation of three recent articles based on the active workspace’s interest profile. Users can read these articles, ask follow-up queries, or reset the session to clear the chat history without triggering new daily recommendations. Switching workspaces resets the session and provides new recommendations tailored to the selected workspace. To ensure diversity, the system avoids recommending the same article multiple times within a session.

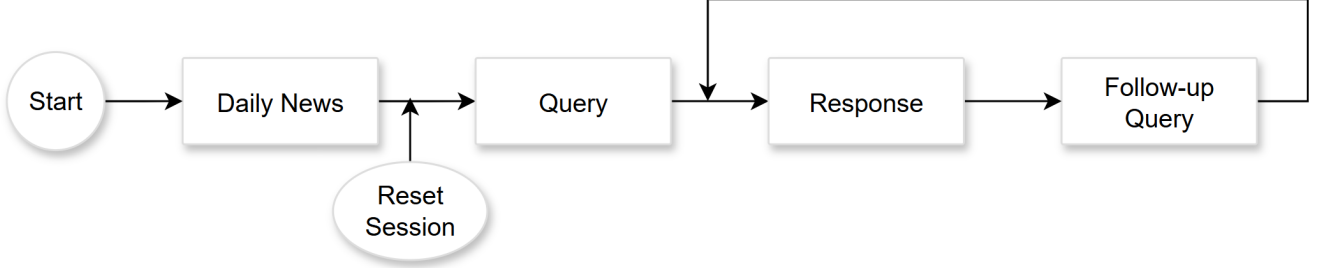


Figure 6: Query flowchart.

5.3 Recommendation Pipeline

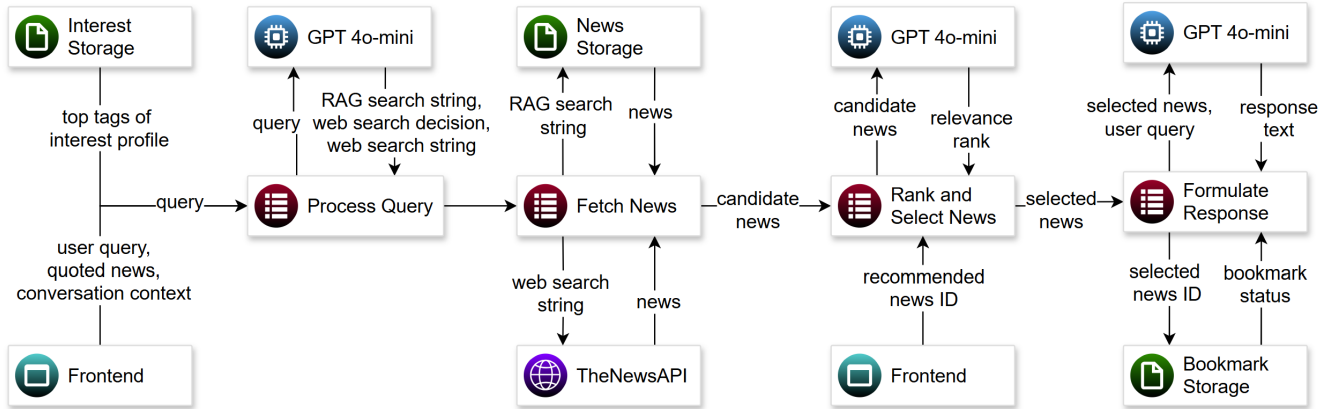


Figure 7: Steps of content recommendation.

5.3.1 Query Processing

The query module, powered by an LLM, processes user input, quoted articles, conversation history, and workspace interest tags. The LLM determines if a web search is needed (e.g., for queries requesting historical news) and generates two search strings: a short web search phrase (1-3 words) and a descriptive RAG query. For example, a query like “history of Tesla” might yield:

- `web_search_required=true`

- `web_search_phrase="Tesla history"`
- `rag_query="Tesla's evolution and historical milestones"`

The LLM's output is parsed using regex to extract these fields. The prompt, detailed in Appendix 9.1.2, instructs the LLM to prioritize semantic understanding, enabling it to interpret vague queries (e.g., "more" after quoting an article) by leveraging context.

5.3.2 News Retrieval

The RAG database stores article summaries as embeddings, generated using a HuggingFace text-to-embedding model. A similarity search retrieves relevant articles based on the RAG query. When no query is provided (e.g., for daily recommendations), recent articles are fetched, and the LLM selects the most relevant ones based on the workspace's interest profile.

For queries requiring news older than one week, the LLM would trigger a web search and generate a concise search phrase. For example, Table 2 shows a case where a user queries follow-up information on post-AGI developments, quoting a recent article. The LLM identifies the need for historical context and generates the phrase "post-AGI insights" to retrieve relevant older articles.

User Query	'this is really interesting, any more on the post-AGI insights?'
Quoting	'Microsoft CEO Satya Nadella claimed Google fumbled its AI opportunity, but DeepMind is already hiring for a post-AGI future'
Web Search Phrase	post-AGI insights

Table 2: Examples of web search phrases.

5.3.3 Response Formulation

The LLM ranks candidate articles (from RAG and web searches) based on relevance to the user's query by prompting (Appendix 9.1.3). Scores are computed as $\frac{1}{\text{rank}}$, normalized to sum to 1, representing the probability of relevance. The top three articles not previously recommended in the session are selected to ensure diversity.

The LLM then generates a concise response summarizing the selected articles and addressing the query, using the prompt in Appendix 9.1.4. Bookmark statuses are retrieved from the SQLite database and included in the response, which is sent to the frontend for display.

5.3.4 Daily Recommendation

The daily recommendation process is triggered at the start of each conversation. The top 10 interest tags of the active workspace are concatenated as a RAG search string to retrieve the top three articles. The LLM summarizes the articles (prompt in Appendix 9.1.5), which is sent to the

frontend for display. This profile-driven process relies solely on the interest profile, without user input or conversation context.

6 User Interface Feature

This section describes the user interface features enhancing interaction with the news recommendation system, including chat controls, bookmark management, and workspace customization. The page structure is illustrated in Figure 8.

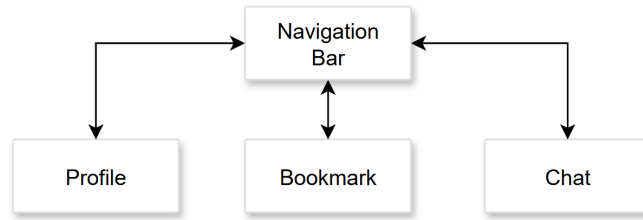


Figure 8: Page structure of user interface.

6.1 Chat Interface

The chat interface provides interactive controls for querying and interacting with recommended articles through intuitive designs.

6.1.1 Quote

Users can quote articles to provide context for follow-up queries, as shown in Figure 9. By clicking the quote button on an article, users include its title and summary in the query, enabling the agent to recommend related news (e.g., additional reports on the same topic). Quoting simplifies user input, as the LLM infers context from the quoted article, reducing the need for lengthy descriptions.

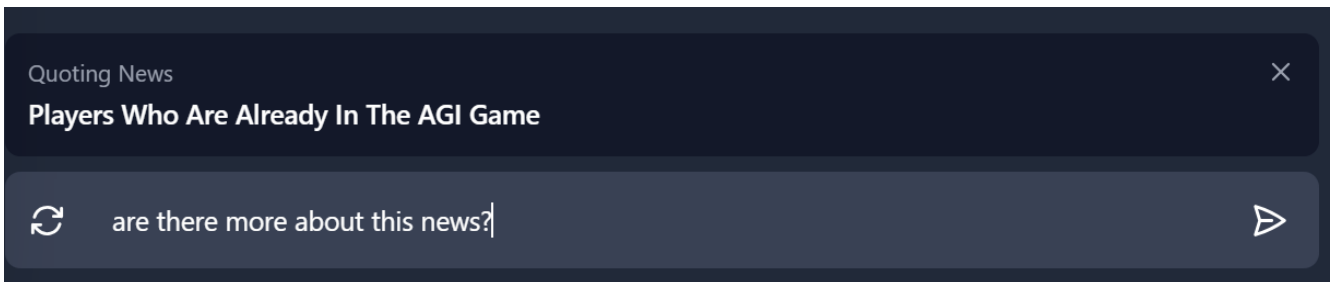


Figure 9: Chatbar with a news article quoted.

6.1.2 Controls

Each article, as depicted in Figure 10, includes a clickable title linking to the full article, a collapsible summary, and buttons for quoting and bookmarking. The bookmark button is toggleable, allowing users to save or remove articles for later viewing. These controls enhance user engagement by facilitating easy navigation and interaction with recommended content.

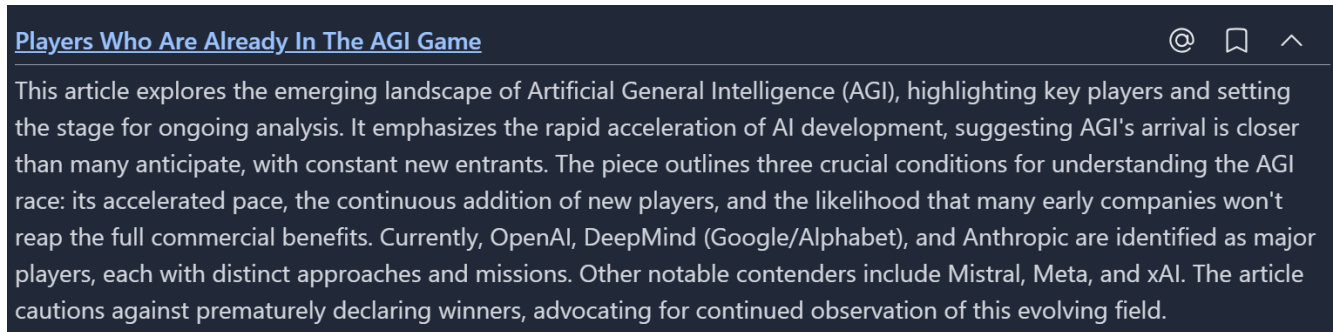


Figure 10: Interface of a news item.

6.2 Bookmarks

The bookmark system enables users to save and manage news articles for later viewing, ensuring accessibility beyond chat sessions.

6.2.1 Feature Description

The bookmark system allows users to save news articles for later reading, preventing loss after chat sessions (Figure 11). On the bookmarks page, articles are displayed in a card format, consistent with the chat interface (Section 6.1), featuring clickable titles linking to full articles, collapsible summaries, and controls for deleting or quoting articles. The pagination feature shows 10 bookmarks per page, enhancing usability for managing reading lists.

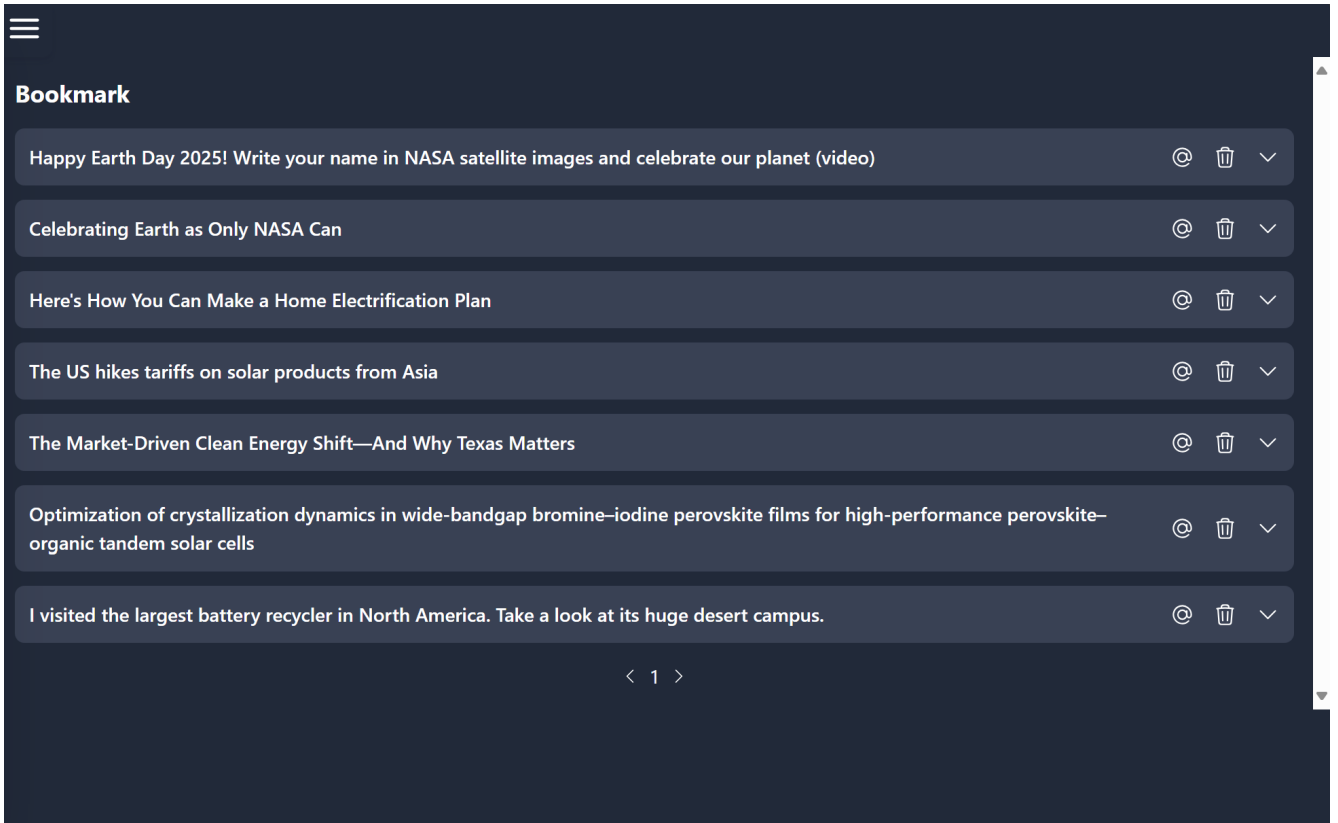


Figure 11: Interface of the bookmark page.

6.2.2 Implementation

The bookmark module, illustrated in Figure 12, manages creation, deletion, and retrieval using SQLite [34]. On the bookmarks page, workspace-specific bookmarks are fetched and displayed in card format, supporting quote and delete interactions. When adding a bookmark, the frontend sends the news ID and user information via API. The module retrieves the full news entry from news storage and saves it with user and workspace IDs (Section 6.3) in the bookmark storage. This retains the news on the system when it exceeds the data retention period. Deletion removes entries using bookmark, user, and workspace IDs.

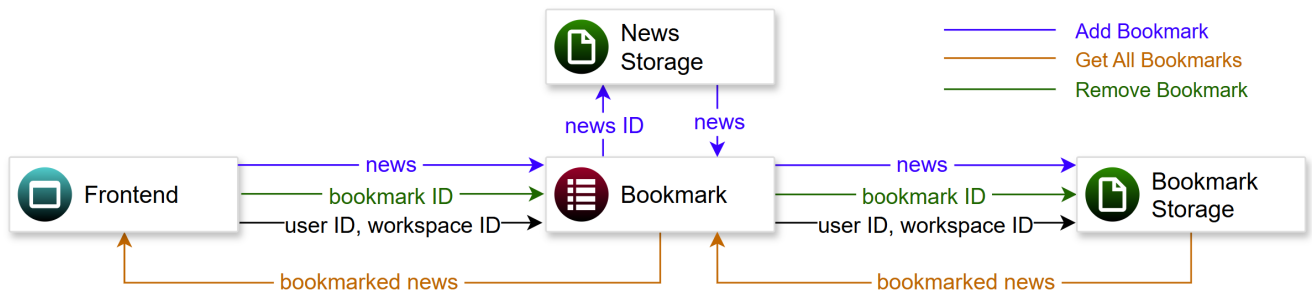


Figure 12: Process of handling bookmarks.

6.3 Workspaces

The workspace system enables users to customize news experiences through independent profiles.

6.3.1 Feature Description

The workspace system enables users to maintain multiple profiles with independent interest tags and bookmarks (Section 6.2), customizing news browsing for specific interests. Users switch workspaces via a collapsible sidebar menu (Figure 13), for smooth transitions across different news contexts.

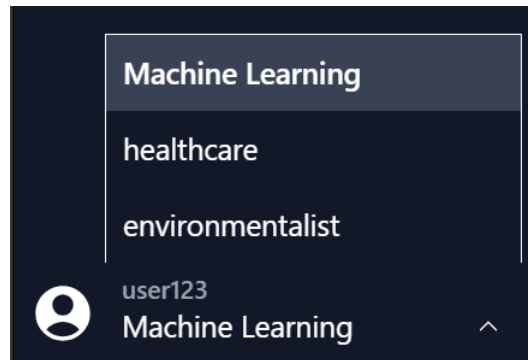


Figure 13: Switching workspace from the sidebar.

Workspace management occurs in the profile page’s workspace section, where profiles are listed by creation time. Users can create new workspaces using the “+ Workspace” button or delete existing ones, enabling flexible profile customization (Figure 14).



Figure 14: Interface for managing workspace.

Each workspace’s interest profile, displayed on the profile page, shows the top 10 tags sorted by importance, derived from user activity (Section 5.1). This enables tailored recommendations, varying by workspace (e.g., AI for “machine learning,” environmental for “environmentalist,” Figure 15, Figure 16).

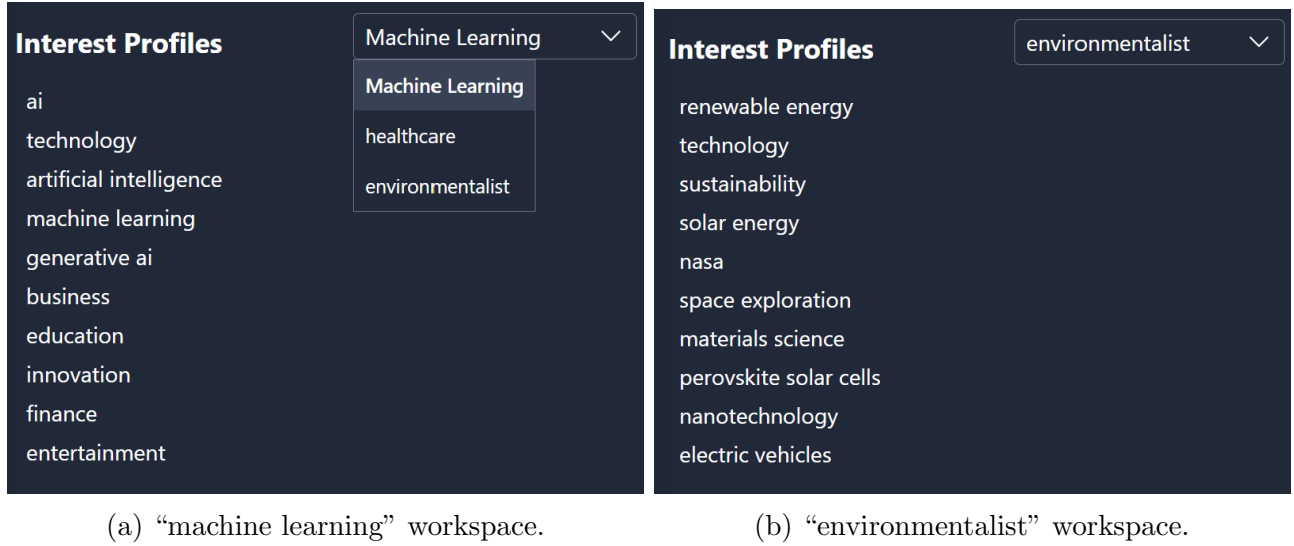


Figure 15: Interest profile of different workspaces.

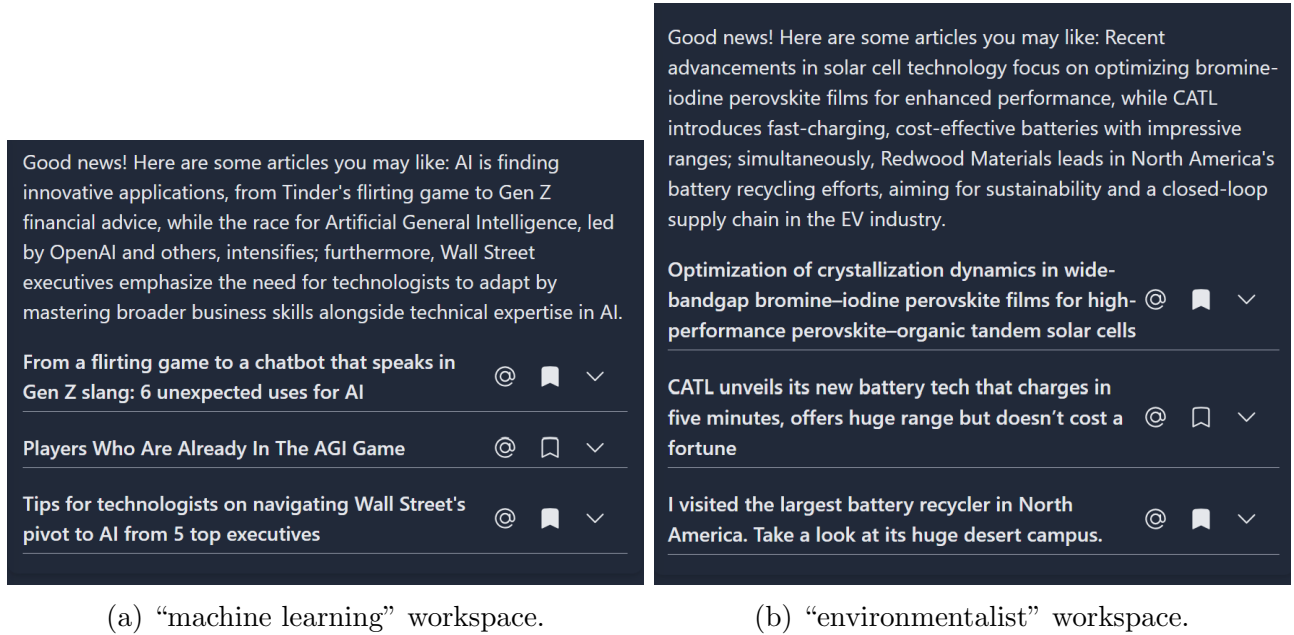


Figure 16: Recommendations in different workspaces.

6.3.2 Implementation

The workspace system associates a workspace ID with tasks (bookmarks, queries, interests), managed via an SQLite database storing the workspace ID, name, and corresponding user ID. It supports create, fetch, and delete operations. The active workspace ID, stored in frontend context, is included in API calls but resets on page refresh or browser close. Interest profiles leverage ChromaDB for tag storage (Section 5.1), enabling personalized recommendations (Section 5).

7 Conclusion

This project successfully developed a personalized news recommendation system using a large language model (LLM) and Retrieval-Augmented Generation (RAG) to deliver tailored news articles. The web application’s intuitive chat interface, workspace profiles, bookmarking, and web search capabilities enhance user engagement and recommendation accuracy. By dynamically updating interest profiles and supporting natural language queries, the system provides a seamless and responsive experience across devices. Future improvements could focus on scaling to multiple users, enhancing tag accuracy for interest profiles, and integrating additional news sources to broaden coverage. This implementation demonstrates the potential of LLM-driven systems for personalized content delivery. Future work could explore a multi-user system to enable feature and user modeling across multiple profiles, enhancing engagement and recommendation accuracy for real-world news delivery.

8 References

- [1] C. Wu, F. Wu, Y. Huang, and X. Xie, “Personalized news recommendation: Methods and challenges,” *arXiv preprint arXiv:2106.08934*, 2021.
- [2] K. Lang, “Newsweeder: Learning to filter netnews,” in *Machine learning proceedings 1995*, pp. 331–339, Elsevier, 1995.
- [3] M. Claypool, A. Gokhale, T. Miranda, P. Murnikov, D. Netes, and M. Sartin, “Combining content-based and collaborative filters in an online newspaper,” in *Proceedings of ACM SIGIR Workshop on Recommender Systems*, 1999.
- [4] A. S. Das, M. Datar, A. Garg, and S. Rajaram, “Google news personalization: Scalable online collaborative filtering,” in *Proceedings of the 16th International Conference on World Wide Web*, pp. 271–280, 2007.
- [5] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, “Grouplens: An open architecture for collaborative filtering of netnews,” in *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work*, pp. 175–186, 1994.
- [6] F. Garcin, K. Zhou, B. Faltings, and V. Schickel, “Personalized news recommendation based on collaborative filtering,” in *2012 IEEE/WIC/ACM International Conferences on Web Intelligence and Intelligent Agent Technology*, vol. 1, pp. 437–441, IEEE, 2012.
- [7] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670, 2010.
- [8] O. Phelan, K. McCarthy, and B. Smyth, “Using twitter to recommend real-time topical news,” in *Proceedings of the 3rd ACM Conference on Recommender Systems*, pp. 385–388, 2009.
- [9] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [10] S. Okura, Y. Tagami, S. Ono, and A. Tajima, “Embedding-based news recommendation for millions of users,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1933–1942, 2017.
- [11] K. Joseph and H. Jiang, “Content based news recommendation via shortest entity distance over knowledge graphs,” in *Companion Proceedings of The 2019 World Wide Web Conference*, pp. 690–699, 2019.
- [12] H. Wang, F. Zhang, X. Xie, and M. Guo, “Dkn: Deep knowledge-aware network for news recommendation,” in *Proceedings of the 2018 World Wide Web Conference*, pp. 1835–1844, 2018.

- [13] T. Qi, F. Wu, C. Wu, Y. Huang, and X. Xie, “Privacy-preserving news recommendation model learning,” *arXiv preprint arXiv:2003.09592*, 2020.
- [14] C. Wu, F. Wu, M. An, J. Huang, Y. Huang, and X. Xie, “Neural news recommendation with attentive multi-view learning,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2214–2222, 2019.
- [15] G. Zheng, F. Zhang, Z. Zheng, Y. Xiang, N. J. Yuan, X. Xie, and Z. Li, “Drn: A deep reinforcement learning framework for news recommendation,” in *Proceedings of the 2018 World Wide Web Conference*, pp. 167–176, 2018.
- [16] X. Li, S. Feng, and X. Zhang, “A review of methods using large language models in news recommendation systems,” in *2024 IEEE/ACIS 27th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pp. 81–86, IEEE, 2024.
- [17] C. Wu, F. Wu, T. Qi, and Y. Huang, “Empowering news recommendation with pre-trained language models,” in *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, pp. 1652–1656, 2021.
- [18] X. Runfeng, C. Xiangyang, Y. Zhou, W. Xin, X. Zhanwei, Z. Kai, *et al.*, “Lkpnr: Llm and kg for personalized news recommendation framework,” *arXiv preprint arXiv:2308.12028*, 2023.
- [19] Q. Liu, N. Chen, T. Sakai, and X.-M. Wu, “Once: Boosting content-based recommendation with both open-and closed-source large language models,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pp. 452–461, 2024.
- [20] Y. Yada and H. Yamana, “News recommendation with category description by a large language model,” *arXiv preprint arXiv:2405.13007*, 2024.
- [21] J. Zhang, R. Xie, Y. Hou, X. Zhao, L. Lin, and J.-R. Wen, “Recommendation as instruction following: A large language model empowered recommendation approach,” *ACM Trans. Inf. Syst.*, Dec. 2024. Just Accepted.
- [22] S. Gao, J. Fang, Q. Tu, Z. Yao, Z. Chen, P. Ren, and Z. Ren, “Generative news recommendation,” in *Proceedings of the ACM Web Conference 2024*, WWW ’24, (New York, NY, USA), p. 3444–3453, Association for Computing Machinery, 2024.
- [23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [24] T. T. Tran, C.-E. González-Gallardo, and A. Doucet, *Retrieval Augmented Generation for Historical Newspapers*. New York, NY, USA: Association for Computing Machinery, 2025.

- [25] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, “Agentic retrieval-augmented generation: A survey on agentic rag,” *arXiv preprint arXiv:2501.09136*, 2025.
- [26] “Newsapi: Search news and blog articles on the web.” <https://newsapi.org>, 2024. Accessed: 2025-05-16.
- [27] “Nltk: Natural language toolkit.” <https://www.nltk.org>, 2024. Accessed: 2025-05-16.
- [28] “Chromadb: Open-source vector database for embeddings.” <https://www.trychroma.com>, 2024. Accessed: 2025-05-16.
- [29] “Huggingface: Transformers: State-of-the-art machine learning for pytorch, tensorflow, and jax.” <https://huggingface.co>, 2024. Accessed: 2025-05-16.
- [30] “Thenewsapi: Real-time news search api.” <https://www.thenewsapi.com>, 2024. Accessed: 2025-05-16.
- [31] F. Goossen, W. IJntema, F. Frasincar, F. Hogenboom, and U. Kaymak, “News personalization using the cf-idf semantic recommender,” in *Proceedings of the International Conference on Web Intelligence, Mining and Semantics*, pp. 1–12, 2011.
- [32] M. Capelle, F. Frasincar, M. Moerland, and F. Hogenboom, “Semantics-based news recommendation,” in *Proceedings of the 2nd international conference on web intelligence, mining and semantics*, pp. 1–9, 2012.
- [33] S. Han, H. Huang, and J. Liu, “Neural news recommendation with event extraction,” *arXiv preprint arXiv:2111.05068*, 2021.
- [34] “Fastapi: High-performance web framework for python.” <https://fastapi.tiangolo.com>, 2024. Accessed: 2025-05-16.

9 Appendix

9.1 Prompts

9.1.1 Generate Summary and Tags

```
prompt = (
    f"Generate a 150-word summary and 5 category tags for the\n"
    f"following news article. "\n"
    f"Focus on the key points from the title and content, keeping it\n"
    f"concise and informative.\n\n"
    f"Title: {title}\n"
    f"Content: {content}\n\n"
    f"Return the summary enclosed by the tags <summary></summary>\n"
```

```

        f"Return the category tags enclosed by <tags></tags> and
        separated by comma. Example: <tags>AI,healthcare,gaming,
        Nintendo,Xbox</tags>\n"
    )

```

9.1.2 Query Processing

```

agent_context = (
    f"You are a news recommendation expert. Your task is to assist in
    finding relevant news articles for a user. You will be provided
    with the user's query, a snippet from a previously recommended
    article the user is referring to (if applicable), the current
    conversation history (if applicable) and the top 5 tags the user
    is interested in (if applicable). "
    f"Based on this information, you must:"
    f"1a. **Determine if a web search for older news is necessary.**
    Consider whether the user's query requires historical context or
    information that might not be included in news within a week."
    f"1b. **Formulate short search phrase for the web search.** This
    should be a short phrase that can be used to search for older
    news articles to answer user's query. It should be kept as short
    as possible, within 1-3 words. "
    f"2. **Formulate a search query for the RAG database.** This query
    should be specific and effective in retrieving relevant articles
    from the database."
    f"Your output should be structured as follows:"
    f"<response>web_search_required=true/false,"
    f"web_search_phrase='search phrase for news', "
    f"rag_query='search query for RAG database'</response>"
)

```

```

prompt = (
    f"user_query='{user_input}'. "
    f"previous_article_snippet='{quote if quote else 'none'}'. "
    f"top_tags={top_tags}."
    f"conversation_history={context if context else 'none'}. "
)

```

9.1.3 News Ranking

```

agent_context = (
    f"You are a news recommendation expert. Your task is to assist in
    finding relevant news articles for a user. You will be provided

```

```

        with the user's query and a list of candidate articles to choose
        from."
f"Based on this information, you must rank all {len(docs)} articles
    from most to least relevant to the user's query. "
f"Your output should be structured as follows:"
f"<response>[index1, index2, ...]</response>"
f"All indices should be included in decreasing order of relevance. "
)

prompt = (
    f"user_query={query}"
    f"articles:\n{article_string}"
)

```

9.1.4 Generate Response to User Query

```

agent_context = (
    f"You are a news recommendation expert. Your task is to curate news
    articles to answer user's query."
    f"You will be provided with the user's query and {retrieve_count}
    articles ."
    f"Your task is to generate a short answer that summarizes the
    selected articles to answer user's query. "
    f"Your output should be structured as follows:"
    f"<response>short answer</response>"
)

prompt = (
    f"user_query={query}"
    f"articles:\n{article_string}"
)

```

9.1.5 Generate Message for Daily Recommendation

```

prompt = (
    f"Generate a short summary in 60 words for the following articles:\n
    \n"
    f"{context}"
    f"Return your response as a single sentence wrapped in <response>
    tags, formatted exactly as '<response>Short answer here.</
    response>'. "
)

```

9.2 Meeting Minutes

Minutes of the 1st Project Meeting

Date	Saturday, March 24, 2025
Time	9:00 am
Place	Online Zoom meeting
Present	Jiazhi ZHENG Bingjun CHEN Linbing XIANG Tsz Ying CHENG Ziliang LIAO Kaiqiang XU Prof. Kai CHEN
Apology	None
Note-taker	Tsz Ying CHENG

1. Approval of minutes

60 minutes.

2. Discussion items

All students shared their work progress in a presentation and received feedback from Kaiqiang and Prof. Chen.

3. Meeting adjournment

The meeting adjourned at 10:00 AM. The next meeting will be held on April 12.

The student is absent from the 2nd meeting, the 2nd meeting's minutes is thus omitted.

Minutes of the 3rd Project Meeting

Date	Saturday, April 19, 2025
Time	9:00 am
Place	Online Zoom meeting
Present	Jiazhi ZHENG Bingjun CHEN Linbing XIANG Tsz Ying CHENG Ziliang LIAO Kaiqiang XU Prof. Kai CHEN
Apology	None
Note-taker	Tsz Ying CHENG

1. Approval of minutes

60 minutes.

2. Discussion items

All students shared their work progress and received feedback from Kaiqiang and Prof. Chen.

3. Meeting adjournment

The meeting adjourned at 10:00 AM. The next meeting will be held on April 26.

Minutes of the 4th Project Meeting

Date Saturday, April 26, 2025
Time 9:00 am
Place Online Zoom meeting
Present Jiazhi ZHENG
Bingjun CHEN
Linbing XIANG
Tsz Ying CHENG
Ziliang LIAO
Kaiqiang XU
Prof. Kai CHEN
Apology None
Note-taker Tsz Ying CHENG

1. Approval of minutes

60 minutes.

2. Discussion items

All students share their work progress and receive feedback from Kaiqiang and Prof. Chen:

- Bingjun Chen completed a functioning github repo system and laid out future work
- Linbing Xiang didn't have new progress this week due to other projects.
- Tszying Cheng completed bookmark and user profile features
- Jiazhi Zheng completed interest-based recommendation and multi-round recommendation and laid out future work

Kaiqiang Xu provided general suggestions on presenting work results.

3. Meeting adjournment

The meeting adjourned at 10:00 AM. The next meeting will be held on May 10.

Minutes of the 5th Project Meeting

Date Saturday, May 10, 2025
Time 9:00 am
Place Online Zoom meeting
Present Jiazhi ZHENG
Bingjun CHEN
Linbing XIANG
Tsz Ying CHENG
Ziliang LIAO
Kaiqiang XU
Prof. Kai CHEN
Apology None
Note-taker Tsz Ying CHENG

1. Approval of minutes

60 minutes.

2. Discussion items

All students share their work progress and receive feedback from Kaiqiang and Prof. Chen:

- Bingjun Chen beautified his frontend.
- Linbing Xiang added natural language Q&A and did a demonstration.
- Tszying Cheng completed workspace system, web search and quoting feature.
- Jiazhi Zheng completed and displayed LLM summarization.
- Zilaing Liao completed text and description-based recommendation and is working on user system.

Prof. Chen and Kaiqiang Xu explained the format of the final presentation and report.

3. Meeting adjournment

The meeting adjourned at 10:00 AM. The next meeting will be held on May 17.

Minutes of the 6th Project Meeting

Date Saturday, May 17, 2025
Time 9:00 am
Place Online Zoom meeting
Present Jiazhi ZHENG
Bingjun CHEN
Linbing XIANG
Tsz Ying CHENG
Ziliang LIAO
Kaiqiang XU
Prof. Kai CHEN
Apology None
Note-taker Tsz Ying CHENG

1. Approval of minutes

60 minutes.

2. Discussion items

All students presented their work and performed live demonstration.

Professor Chen announced the submission details of the final report, it shall include the report, presentation slides and meeting minutes.

3. Meeting adjournment

The meeting adjourned at 10:10 AM. This is the last meeting of the project.